JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

Project Interim Report

Name	Biswajeet Pradhan
USN	241VMTR01011
Elective	Data Analytics (Integrated with IOA, UK)
Date of Submission	28-05-2026

A Presentation or a PDF file of the Research Methodology covering the following points should be uploaded on ERP:

➢ Objectives of the Study:

The main reason for conducting this study is to bridge the gap between advanced data science and everyday business management. Social media has changed the way customers interact with brands, turning platforms like **Twitter (X)** into public feedback forums where any negative post can quickly snowball and damage a company's reputation. By building this system, we want to prove that airlines don't have to just react to customer outrage. Instead, they can use machine learning to systematically monitor customer feedback, understand their pain points in real-time, and make data-driven decisions to keep their passengers happy and protect their brand image.

12 specific objectives to achieve:

1. **Protecting real-time brand reputation:** Providing airlines with a system to catch public complaints immediately, preventing them from turning into viral PR disasters.
2. **Pinpointing operational failures:** Automatically identifying exactly *what* is making customers angry (such as lost luggage, flight delays, or cancellation policies) so managers can address the root cause.
3. **Saving manual support hours:** Eliminating the need for customer service teams to manually read, sort, and tag thousands of messy social media posts every day.
4. **Identifying customer service bottlenecks:** Highlighting internal organizational delays, such as long phone hold times, allowing management to improve support response rates.
5. **Analyzing temporal peak-stress periods:** Finding out if complaints spike during specific hours of the day or days of the week, which helps airlines schedule their support staff more effectively.
6. **Comparing sentiment across carriers:** Benchmark different airlines to see how they perform in terms of customer satisfaction and positive-to-negative feedback ratios.

7. **Optimizing company resource allocation:** Helping management decide where to invest resources (e.g., hiring more ground crew for bags vs. expanding telephone support lines) based on actual feedback data.
8. **Capturing raw and honest customer feedbacks:** Utilizing social media reviews, which are much more candid, immediate, and honest than traditional email satisfaction surveys.
9. **Routes issues to the right departments:** Classifying the reasons for negative feedback so that baggage issues go straight to the luggage team, and booking issues go to the ticketing team.
10. **Building a practical business tool:** By turning complex mathematical algorithms (like XGBoost or LightGBM) into a simple, usable Flask website that any non-technical manager can operate.
11. **Providing concrete metrics for executives:** Generating statistically backed sentiment reports that executives can use to monitor the airline's overall customer experience health.
12. **Increasing customer retention:** Responding to customer issues faster and more accurately, showing passengers their complaints are heard, which improves long-term brand loyalty.

➢ Scope of the Study

When defining the boundaries of our project, it was necessary to ensure that it is built over a highly focused, stable, and practical machine learning solution. The scope is designed to balance advanced data science techniques with the reality of building a local web application. It specifies exactly what data we are processing, what algorithms we are optimizing, and where to draw the line to keep the project manageable. By setting these clear limits, it was ensured that the model remains highly accurate and specialized for the airline industry without becoming bogged down by excessive computing requirements.

1. **Focusing on US airline customer feedback:** Restricting the data source to a structured, secondary dataset of over 14,000 tweets aimed at six major United States airlines.
2. **Analyzing English-only textual reviews:** Processing only English text feedback, which keeps multilingual translation and multilingual sentiment analysis outside the study.

3. **Cleaning social media noise:** Filtering out useless micro-blogging noise like web links, user tags, and special characters, while transforming emojis into text formats.
4. **Vectorizing text using TF-IDF:** Converting cleaned text reviews into numerical matrices using TF-IDF vectorization to capture unigram and bigram word importances.
5. **Evaluating sparse and dense configurations:** Comparing sparse TF-IDF outputs against dense representations using Truncated SVD (LSA) to optimize memory and speed.
6. **Optimizing with GPU-accelerated algorithms:** Tuning three advanced gradient boosted decision trees—XGBoost, LightGBM, and CatBoost—using Optuna and GPU acceleration.
7. **Resolving dataset class imbalance:** Implementing automated class-weighting and validation monitors to prevent the models from ignoring positive and neutral tweets.
8. **Developing a local Flask application:** Packaging the finished vectorizer, column transformers, and final classifier into a local web application.
9. **Excluding real-time data streaming:** Relying entirely on static batch files for training, meaning live Twitter API integration or real-time web scraping is excluded.
10. **Avoiding sarcasm and irony detection:** Sticking to standard supervised classification, meaning complex deep-learning contextual models for detecting sarcasm are left out.
11. **Bypassing multimedia and visual processing:** Focusing strictly on text-based NLP, meaning attached images or videos of broken bags or flight boards are ignored.
12. **Measuring performance with standard evaluation metrics:** Using accuracy, precision, recall, and confusion matrix heatmaps to benchmark the models on the holdout test set.

➢ Methodology

1. **Ingesting and Preprocessing the Raw Data :** The pipeline starts by loading the raw serialized Twitter dataset and dropping non-informative columns like user IDs, usernames, and tweet dates. Social media text is notoriously noisy, so

first it is cleaned up using a custom **text-processing** pipeline. Then, comes the **regular expressions** to strip out hyperlinks and Twitter handles, but it uses a **demojization** function to convert graphical emojis into text words (like turning a sad face into the word "sad") so the pipeline do not lose critical emotional cues. After this initial cleaning, we **tokenize** the text into individual words, filter out common stop-words, and apply part-of-speech-aware **lemmatization** to convert words back to their dictionary **root** forms (for example, turning "flying" and "flew" to "fly").

2. **Scaling and Vectorizing the Features :** Once the text is cleaned, we combine multiple text columns—specifically the airline name, timezone, weekday, customer review, and complaint reason—into a single, comprehensive text string. To handle the numerical features, we apply **MinMaxScaler** to normalize the hour of the tweet, and **StandardScaler** to scale the retweet counts and complaint confidence scores. Next, we convert the combined text feature into numbers using a **TF-IDF vectorizer** that captures both single words and two-word combinations (**unigrams** and **bigrams**), using sublinear scaling to prevent highly frequent words from dominating the feature space. Finally, we convert this vectorized matrix into a DataFrame and prefix all the column names with "tfidf_" to prevent duplicate column name errors when merging with our numerical data.
3. **Training and Tuning the Ensemble Models :** With our features fully prepared, we partition the dataset into training, validation, and testing sets using a standard **80/20 train-test split**. To find the best possible classifier, we set up a hyperparameter search using the **Optuna** framework. Inside this search loop, we train three advanced gradient boosted decision tree ensembles—**XGBClassifer**, **LGBMClassifer**, and **CatBoostClassifer**—running them on a GPU to maximize processing speeds. We implement early stopping callbacks in each model's training loop to monitor the validation set and halt training the moment the accuracy plateaus, which keeps the models from overfitting. We also incorporate automated class weights to handle the severe dataset imbalance, where negative reviews heavily outnumber positive and neutral ones.
4. **Evaluating and Deploying the Pipeline :** After the Optuna search identifies the winning model and best parameters, we evaluate its final performance on the holdout test set. We analyze the model's reliability using multiple

classification metrics, including Overall **Accuracy**, Weighted **Precision**, Weighted **Recall**, and **Confusion Matrix** Heatmaps, to ensure it generalizes well to unseen data. Once we verify that the model is performing successfully **(achieving over 93% accuracy)**, we serialize the trained classifier, the TF-IDF vectorizer, and the column transformers into a single pickle file. Finally, we build a local Flask web application that loads this pickle file, allowing brand managers to type in new feedback and receive instant sentiment predictions in real-time.

➢ Research Design

- **Determining the Nature of the Research :** This study uses a hybrid research design that combines quantitative data analysis with practical, application-based software engineering. Because we are looking at historical social media reviews to categorize them into positive, negative, and neutral classes, this project is descriptive and predictive in nature. It is also an experimental research study because we are comparing the performance of three different advanced machine learning models—**XGBoost, LightGBM**, and **CatBoost**—to find out which algorithm is the most accurate at classifying airline customer feedback. Rather than relying on guesswork, we use empirical data (over **14,000** tweets) to run these tests, ensuring our findings are backed by solid statistics.

- **Identifying the Variables of the Study :** Our research design revolves around mapping the relationships between the independent variables (the features we use as inputs) and the dependent variable (the target we want to predict). The dependent variable in this study is the passenger's sentiment, which is labeled as negative, neutral, or positive. The independent variables consist of two types: textual features and numerical metadata. The textual features are the high-dimensional TF-IDF vectors that represent the words and phrases customers used in their reviews. The numerical features include metadata like the time the tweet was posted (hour), the confidence level of the logged complaint reason (reason_confidence), and the viral reach of the post (retweets). By analyzing these variables together, the models learn to spot the complex patterns that link specific customer complaints to overall sentiment.

- **Outlining the Experimental and Procedural Setup :** We structured the procedural design of this study as a step-by-step pipeline, moving from raw data ingestion to a functional local web deployment. To test our models fairly, we use a standard holdout split, dividing our data into training, validation, and testing sets. We use the training and validation sets to run our GPU-accelerated Optuna parameter search, which automatically tests different settings for tree depth, learning rates, and estimator counts. Once the optimal model is selected and verified on the test set, the final stage of our design involves serializing the entire preprocessing and classification pipeline. This serialized file is integrated into a local Flask web interface, turning our mathematical research findings into a practical, real-time brand management tool.

➢ Data Collection Method

Instead of setting up real-time web scrapers or struggling with the technical restrictions and high costs of the Twitter API, this project relies entirely on **secondary data collection**. We utilize a pre-existing, highly structured historical dataset consisting of over 14,000 individual customer tweets directed at major airlines. Choosing a high-quality secondary dataset is a practical decision because it allows us to bypass the complex, time-consuming data gathering phase and focus all of our efforts on data cleaning, feature engineering, model optimization, and deployment. It also gives a stable, standardized benchmark that makes it easy to compare different machine learning models fairly.

The dataset contains customer feedback directed at six major United States airlines: **United, US Airways, American, Southwest, Delta,** and **Virgin America**. Beyond the raw text of the customer feedback, this dataset provides a rich, multi-dimensional variety of features. These attributes include the specific airline tagged, the day of the week, the geographical timezone of the user, the hour the tweet was posted, and the retweet count. Most importantly, for negative feedback, the dataset includes categorized reasons for the complaints (such as "Late Flight", "Lost Luggage", or "Customer Service Issue"). This rich metadata allows us to train our models not just on raw words, but on the real-world operational context of the complaints.

For supervised machine learning to work, our models need a highly reliable target label to learn from. This dataset includes sentiment labels classifying each tweet as positive, negative, or neutral, along with a confidence score for those classifications. Having verified labels is an absolute necessity because it gives our algorithms a reliable ground-truth baseline.

➢ Data Analysis Tools

- **Manipulating and Preprocessing the Data :** Python serves as the programming foundation for our entire project. To manipulate data frames and perform array operations efficiently, we use Pandas and NumPy—specifically leveraging the cudf.pandas extension to accelerate our data frame tasks directly on the GPU. For cleaning up noisy text, we use the cleantext library to lowercase words and remove punctuation, alongside the emoji library to translate raw emojis into clean text tokens. For core NLP preprocessing, we rely on the Natural Language Toolkit (NLTK), utilizing its tokenization tools to split reviews into words, its built-in stop-words corpus to filter out filler words, and the WordNetLemmatizer to normalize different word endings to their root forms.

- **Transforming and Scaling the Features :** To convert our processed text and numbers into a structured format that machine learning algorithms can read, we use Scikit-learn. We utilize TfidfVectorizer to capture unigram and bigram word importances from the text, using its sublinear scaling feature to prevent high-frequency words from distorting the results. For handling our numerical columns, we use Scikit-learn's MinMaxScaler (to scale tweet hours) and StandardScaler (to normalize retweet counts and complaint confidence scores). We group these transformers together using Scikit-learn's ColumnTransformer and combine them with SciPy's sparse matrix tools, allowing us to build a clean, combined feature dataset without consuming massive amounts of computer memory.

- **Visualizing and Evaluating Model Performance :** To measure how well our final model generalizes to unseen data, we rely on Scikit-learn's metrics library alongside visual plotting libraries. We use **Matplotlib** and **Seaborn** to plot our

results, specifically creating **Confusion Matrix Heatmaps** that compare our true labels against predicted labels. This matrix is a critical tool because it lets us see exactly where the model makes mistakes, such as showing the "adjacent class confusion" between positive and neutral sentiments. To get a complete picture of model performance, we calculate all four core metrics: **Accuracy** (the overall percentage of correct predictions), **Precision** (measuring our rate of false positives), **Recall** (tracking our rate of false negatives), and the **F1-Score** (which combines precision and recall into a single score). Using all four metrics is essential because it ensures our model is performing reliably across all categories, despite the heavy class imbalance where negative reviews outnumber positive ones.